AI IN CODING

Senior sam masterclass
Coding, Prompt Engineering, Automation

: The New Era of "Vibe Coding"

1. The Big Lie

For decades, we were told: "To build an app, you need to be a math genius." You need to memorize semicolons, curly brackets, and complex algorithms. **That is no longer true.**

2. What Changed?

Artificial Intelligence (AI) has democratized code. Just as calculators meant you didn't need to do long division in your head, AI means you don't need to type code by hand. We have entered the era of **"Vibe Coding."**

- **Old Way:** You type the code. You fix the syntax. You worry about the comma.
- **New Way:** You manage the *logic* (the Vibe). You tell the AI *what* to build, and it handles the *how*.

3. Who is this Guide For?

This guide is for the visionary, the entrepreneur, and the problem solver who has ideas but lacks the technical skills. By the end of this document, you will not just be "generating code"—you will be building functional, sellable software products.

The AI Tool Stack

1. The Brain (The LLM)

You need a smart AI model. Not all AIs are equal for coding.

- **Claude 3.5 Sonnet:** Currently the "King of Code." It follows instructions perfectly and rarely makes logic errors.
- **ChatGPT (GPT-4o):** Excellent for explaining concepts and debugging errors.

2. The Editor (The Workshop)

You need a place to write the code.

- **VS Code (Visual Studio Code):** The industry standard. It's free, light, and works on any computer.
- **Cursor:** A new editor that has AI built *inside* it. It can write code for you as you type. (Highly recommended for beginners).

3. The Server (The Engine)

Your code needs a home. We will use **XAMPP** (explained in Page 5). It turns your laptop into a powerful server, allowing you to run PHP and MySQL entirely offline.

Charter 3: The Art of Prompt Engineering – Speaking the Machine's Language

1. The Core Philosophy: You Are the Architect

Most beginners fail at AI coding because they treat the AI like a magic wand. They say, *"Abra Kadabra, make me an app,"* and they get garbage in return.

To succeed, you must shift your mindset.

- **You are the Architect:** You understand the vision, the logic, and the flow.
- **The AI is the Contractor:** It creates the bricks (code) and lays them down.

If the Architect gives the Contractor a vague drawing on a napkin, the house will collapse. If the Architect gives a detailed blueprint, the house will stand. **Prompt Engineering is simply the skill of drawing detailed blueprints with words.**

2. The "C.R.E.A.T.E." Framework

Never write a one-sentence prompt. Use the **C.R.E.A.T.E.** framework to ensure the AI creates exactly what you need on the first try.

C — Context

Who is this for? What is the project?

- *Weak:* "Make a form."
- *Strong:* "I am building a **School Management System for a high school in Kenya**. The system needs to be lightweight because the internet connection is slow."

R — Role

Who should the AI pretend to be?

- *Weak:* "Write code."
- *Strong:* "Act as a **Senior Full-Stack PHP Developer** with 10 years of experience in security and database optimization."

E — Explicit Task

What exactly do you want the AI to do right now?

- *Weak:* "Make it work."
- *Strong:* "Write a **PHP script** that connects to a MySQL database and inserts a new student's name, admission number, and grade."

A — Approach (Constraints)

What tools or rules must the AI follow?

- *Weak:* "Use whatever."
- *Strong:* "Use **Tailwind CSS** for styling. Use **PDO (PHP Data Objects)** for the database connection to prevent SQL injection attacks. Do not use raw SQL queries."

T — Tone & Format

How should the output look?

- *Weak:* "Show me."
- *Strong:* "Provide the code in a **single block**. Add **comments** to every line explaining what the code does so I can learn."

E — Examples (Optional but Powerful)

Give the AI a reference.

- *Strong:* "The style should match the 'Dashboard' layout we created in the previous step."

---

3. The "Bad vs. Good" Showdown

Let’s look at a real-world example of how a better prompt changes the result.

❌ **The Bad Prompt:**

"Write me a login page for my website."

Why this fails: The AI will give you generic, ugly code. It might use outdated HTML, it won't be secure, and it won't look like *your* brand. You will spend hours trying to fix it.

✅ **The "C.R.E.A.T.E." Prompt:**

"I am building a portal for a real estate company **(Context)**. Act as a Front-End Expert **(Role)**. Create a login form HTML and CSS **(Task)**. Use Tailwind CSS for styling. The background should be a soft gray, and the button should be deep blue. Ensure the form is centered on the screen **(Approach)**. Comment the code so I know where to paste my logo **(Format)**."

Why this wins: You get a professional, mobile-responsive, branded login page in 10 seconds.

---

4. The Golden Rules of AI Interaction

Rule #1: One Logic, One Prompt

Do not ask the AI to build the entire app in one go. It will hallucinate (make mistakes). Break it down:

1. First Prompt: Create the Database SQL.
2. Second Prompt: Create the HTML Form.
3. Third Prompt: Create the PHP script to connect them.

Rule #2: The "Refinement" Loop

The first result is rarely perfect. Treat it like a conversation with a human developer.

- *"That looks good, but can you make the 'Submit' button larger?"*
- *"I'm getting an error on line 12. Review the code and fix it."*

- *"Can you rewrite the PHP section to be more secure?"*

Rule #3: Don't guess—Ask the AI to explain

Since you are learning while building, always add this phrase to the end of your prompts:

"After the code, please explain in simple English how this script works and where I should save this file."

---

5. Magic Phrases for Non-Coders

Memorize these "Magic Phrases" to get out of sticky situations:

1. **"Debug this error:"** (Paste the error code).
2. **"Explain like I'm 12:"** (Use this when the AI uses complex jargon).
3. **"Add defensive coding:"** (This tells the AI to add security layers to prevent hackers).
4. **"Optimize for speed:"** (This makes your application run faster).

Summary of Charter 3

Prompt engineering is not about being technical; it is about being **specific**. The AI is a powerful engine, but you are the steering wheel. If you steer vaguely, you crash. If you steer with precision using the C.R.E.A.T.E. framework, you will reach your destination.

4: Iterative Building – The "Lego" Method

1. The Mistake: The "One Big Prompt"

Beginners try to do everything at once: *"Build me a school system."* The AI will try, but it will fail. It will give you a confused mess.

2. The Solution: Modular Construction

Building an app is like building a house with Legos. You build one brick at a time.

- **Step 1:** Build the Database Table. (Make sure it exists).
- **Step 2:** Build the HTML Form. (Make sure you can see it).
- **Step 3:** Build the PHP Script. (Make sure it connects).
- **Step 4:** Test. (Make sure it saves).

3. The "Rule of One"

Never leave a step until it works. Do not try to style the page if the button doesn't work. Do not try to add a "Forgot Password" feature if the "Login" feature is broken. **Solve one problem completely before moving to the next.**

Page 5: Setting Up Your Digital Workshop – Where the Magic Happens

1. Why Can't I Just Open the File?

If you create an HTML file (like a resume), you can double-click it, and it opens in Chrome. Easy.

But **PHP is different**. PHP is a "Server-Side" language. If you just double-click a PHP file, your browser will just show you the code, not the result.

To run PHP, your computer needs to pretend to be a web server.

- **The Problem:** You don't have a server.
- **The Solution:** A free tool called **XAMPP**.

2. Meet Your New Best Friend: XAMPP

XAMPP is a software bundle that installs everything you need in one click. It turns your laptop into a professional development environment.

- **X** (Cross-Platform)
- **A** (Apache) – The Web Server (The Engine).
- **M** (MySQL) – The Database (The Filing Cabinet).
- **P** (PHP) – The Coding Language (The Brain).
- **P** (Perl) – (We don't need this).

3. Step 1: Download & Install

1. Go to Google and search for **"Download XAMPP"**.
2. Click the first link (usually *apachefriends.org*).
3. Download the version for your computer (Windows, Mac, or Linux).
4. Run the installer.
    - **Tip:** When it asks "Select Components," just leave everything checked and click **Next**.
    - **Tip:** Install it in the default location (C:\xampp on Windows). Do not change this unless you have to.

4. Step 2: The Control Panel (The Cockpit)

Once installed, open the **XAMPP Control Panel**. You will see a list of modules. You only care about the top two:

1. **Apache:** This is the web server. Click **Start**.
2. **MySQL:** This is the database. Click **Start**.

Success Check: When the text background turns **Green**, you are live. Your computer is now a server.

**

5. Step 3: The "Magic Folder" (htdocs)

This is the most important concept to understand.

You cannot just save your PHP files on your Desktop or Documents folder. The server won't look there.

You must save your files in the **htdocs** folder.

- **Windows:** Go to C:\xampp\htdocs
- **Mac:** Go to /Applications/XAMPP/xamppfiles/htdocs

The Analogy:

Think of htdocs as the **"Public Display Window"** of a shop. If you put a product (file) in the back room (Desktop), customers (the browser) can't see it. You *must* put it in the window (htdocs).

6. Step 4: Your First "Vibe Check"

Let's prove it works.

1. Open your text editor (VS Code, Cursor, or Notepad).
2. Paste this code:

PHP

```
<?php

echo "I am officially a Developer!";

?>
```

3. **Crucial Step:** Save this file **inside** your htdocs folder. Name it test.php.
4. Open your web browser (Chrome/Edge).
5. In the address bar, type: localhost/test.php and hit Enter.

The Result:

If you see the text **"I am officially a Developer!"**, congratulations. You have successfully set up a local server environment. You are ready to build.

7. Common Setup Headaches

- **"It won't start!"**
  - If the "Apache" module won't turn green, it's usually because another program (like Skype or VMware) is blocking the "Port."
  - *The Fix:* Close Skype completely and try clicking Start again.
- **"Browser says 'Not Found'"**
  - Did you type localhost correctly?
  - Did you save the file in htdocs?
  - Did you name the file test.php.txt by accident? Make sure it ends in .php.

Summary of Charter 5

This is the only "boring" technical part of the journey. Once XAMPP is running, you rarely have to touch it again. You just turn it on, write code in htdocs, and view it at localhost.

Charter 6: Understanding the Blueprint – The Database

1. The Excel Analogy

Most people think "Databases" are complex mathematical grids. They aren't.

If you know how to use Microsoft Excel, you already understand 90% of databases.

- **A Database** is just a Workbook (a file).
- **A Table** is just a Sheet (a tab).

- **A Column** is just a Header (e.g., "Name", "Price").
- **A Row** is just a specific entry (e.g., "John Doe", "KES 500").

The only difference? You don't type data directly into the database. You use your PHP "Waiter" to put it there for you.

2. Anatomy of a Table

When designing a system, don't think about code. Think about **Tables**.

If you are building a School System, what "Sheets" (Tables) would you need?

1. **Users Table:** (To store login info)
2. **Students Table:** (To store grades)
3. **Teachers Table:** (To store staff info)

The Golden Rule: The "ID"

Every single table you ever create **MUST** have a column called id.

- This is a unique number for every row (1, 2, 3...).
- It is the "Primary Key."
- Without it, the computer cannot tell the difference between "John Smith" (Student A) and "John Smith" (Student B).
- **Always** tell the AI: *"Make sure the ID column is Auto-Incrementing."* (This means the database counts 1, 2, 3 automatically).

3. Creating the Blueprint with AI

You don't need to know how to write SQL code. You just need to describe the "Sheet."

The Prompt:

"Act as a Database Architect. I need the SQL code to create a table named students for my MySQL database.

It should have the following columns:

1. id (Auto-increment, Primary Key)
2. full_name (Text)
3. admission_number (Text, must be unique)
4. grade_level (Integer)
5. parent_phone (Text)
6. created_at (Timestamp, defaults to current time)

Provide just the SQL code so I can paste it into phpMyAdmin."

4. Where to Paste the Code (phpMyAdmin)

You have the code from the AI. Now, where does it go?

1. **Open XAMPP:** Make sure Apache and MySQL are Green.
2. **Go to Browser:** Type localhost/phpmyadmin.

3. **Create Database:** Click "New" on the left sidebar. Name it (e.g., school_system) and click "Create."
4. **The SQL Tab:** Click the "SQL" tab at the top of the screen.
5. **Paste & Go:** Paste the code the AI gave you into the big white box and click "Go."

Success: You will see a green checkmark. Your table now exists!

5. Connecting Tables (Relational Data)

This is an "Advanced" tip that adds huge value to your guide.

Real apps connect data. For example, a "Grade" belongs to a "Student."

The Prompt for Relations:

"Now, I need a table called exam_results. It needs to store the marks for Math, English, and Kiswahili.

Crucial: It needs a column called student_id that links this result to a specific student in the students table. Write the SQL to create this table."

6. Common Database Mistakes

- **Spaces in Names:** Never name a column First Name. Computers hate spaces. Use first_name (snake_case).
- **Wrong Data Types:** Don't store a Phone Number as a "Number" (Integer). Why? Because 0722... will become 722... (it removes the leading zero). Always store phone numbers as "Text" (VARCHAR).
- **Forgetting the Database Name:** In your db.php file (from Page 7), if you named your database school_system here, you **must** use that exact name in your code.

Summary of Charter 6

The database is the foundation of your house. If you get the columns right, the coding is easy. If you get the columns wrong, the coding is a nightmare. Spend 80% of your time planning your tables on a piece of paper before you ask the AI to build them.

Charter 7: Building with PHP – The Server-Side Brain

1. The Waiter Analogy

To build apps without coding, you must understand *what* PHP actually does so you can ask for it correctly.

Imagine a Restaurant:

- **HTML (The Menu):** This is what the customer looks at. It creates the buttons and forms.
- **The Database (The Kitchen):** This is where the ingredients (data) are stored.
- **PHP (The Waiter):** The customer cannot walk into the kitchen. They need a waiter to take their order to the kitchen and bring the food back.

PHP is your Waiter. It is the "brain" that moves data between the user's screen and your database. Without it, your "Login" button is just a painting—it doesn't actually *do* anything.

2. Step 1: The "Handshake" (Connecting to the Database)

Before you build anything, your PHP code needs permission to talk to your database. This is called a **Connection File**. You usually create this once and re-use it everywhere.

The "Vibe Coding" Prompt:

"Act as a PHP Developer. Write a file named db.php that connects to a MySQL database named 'school_system'. Use **PDO (PHP Data Objects)** for the connection because it is more secure.

Important: Since I am using XAMPP on localhost, define the server as 'localhost', user as 'root', and leave the password empty. Add a 'Try-Catch' block so if the connection fails, it tells me why."

- **Why this works:** You specified the tool (PDO) and the environment (XAMPP). The AI will give you perfect connection code.
- **Instruction:** Save the code it generates as db.php in your folder.

3. The Big 4: Master "CRUD" and You Master Everything

90% of all applications—from Facebook to a simple To-Do list—are just four operations repeated over and over. In the industry, we call this **CRUD**.

If you can prompt AI to do these four things, you can build *any* app.

C - Create (Insert Data)

Used for: Sign-ups, posting a status, adding a product.

The Prompt:

"I have an HTML form with fields: name, email, and password. Write a PHP script called signup_process.php.

1. Include the db.php file at the top.
2. Check if the form was submitted using $_POST.
3. Insert the user's data into the 'users' table using **Prepared Statements** (to prevent SQL injection).

4. If successful, redirect the user to dashboard.php."

R - Read (Fetch Data)

Used for: Viewing profiles, reading news feeds, seeing a product list.

The Prompt:

"I need to display a list of all registered students. Write a PHP script that:

1. Connects to the database.
2. Selects all columns from the 'students' table.
3. Displays the data in a neat HTML Table using a foreach loop.
4. Add a 'Delete' button next to each student (we will activate this later)."

U - Update (Edit Data)

Used for: Changing passwords, editing a profile, updating inventory.

The Prompt:

"I need to edit a user's details. Create a script that:

1. Fetches the user's current data based on their ID (from the URL).
2. Fills that data into an HTML form so I can see what I'm editing.
3. Writes a separate 'UPDATE' query to save the changes to the database when I click 'Save'."

D - Delete (Remove Data)

Used for: Deleting comments, removing users.

The Prompt:

"Write a simple PHP script that takes an id from the URL (GET method). Run a specific SQL command to DELETE the user with that ID. After deleting, redirect the page back to the main list."

4. The "Security Shield" (Crucial!)

As an AI Developer, you have one major responsibility: **Don't build hackable apps.**

Beginners often use "Raw SQL," which is dangerous. You must force the AI to use **Prepared Statements**.

- **Unsafe Prompt:** "Put the username into the database."
- **Safe Prompt:** "Insert the username into the database using **Prepared Statements and Parameter Binding** to prevent SQL Injection."

Note: You don't need to know how to write parameter binding yourself—you just need to know to ASK for it. The AI will handle the syntax.

5. Troubleshooting PHP with AI

When things break (and they will), PHP usually gives you a "Fatal Error" or a blank white screen.

How to fix it:

1. **Blank Screen?** Ask AI: *"My PHP page is blank. Write a snippet of code I can put at the top of my file to 'Turn on Error Reporting' so I can see what is wrong."*
2. **Fatal Error?** Copy the error line exactly.
    - *Prompt:* "I am getting this error: Parse error: syntax error, unexpected token ";" on line 14. Here is my code. Fix it."

Summary of Charter 7

PHP is just the logic layer. Your job is not to memorize mysqli_query vs PDO::prepare. Your job is to define the **Input** (Form), the **Action** (Create/Read/Update/Delete), and the **Destination** (Database). If you can describe the flow, the AI can write the PHP.

Charter 8: The "Fix-It" Loop – Debugging Without Tears

1. The Golden Rule: Errors Are Clues, Not Failures

When you see a screen full of red text, your instinct might be to panic. Stop.

In the world of coding, an error message is not the computer yelling at you. It is the computer **helping** you. It is telling you exactly what is wrong and exactly where to find it.

Your job is not to *know* how to fix it. Your job is to be the **Messenger** who carries the error from your screen to the AI.

2. The "Error Paste" Protocol

When something breaks, follow this exact 3-step ritual. Do not deviate.

Step 1: Capture the Evidence

Don't just say "It's not working."

- Copy the **Error Message**: (e.g., Fatal error: Uncaught TypeError... on line 45)
- Copy the **Code Context**: Copy the 20 lines of code *around* the line mentioned in the error.

Step 2: The "Fix-It" Prompt

Paste this into your AI tool:

"I am getting the following error in my PHP script:

[PASTE ERROR MESSAGE HERE]

Here is the code snippet around that line:

[PASTE CODE HERE]

Please analyze why this is happening. Explain the mistake simply, and provide the corrected code block. Tell me exactly what to delete and what to add."

Step 3: The Implementation

- Read the AI's explanation. (This is how you learn!)
- Replace the broken code with the new code.
- Refresh your browser.

3. The "Ghost" Bugs (Logic Errors)

Sometimes, there is no red error message. The page just loads... and nothing happens. Or the database stays empty. These are "Logic Errors."

How to catch a Ghost:

You need to ask the AI to play Detective.

The Prompt:

"I am trying to submit this form, but nothing is happening. No error message appears, but the data is not saving to the database.

Here is my form.html code: [PASTE]

Here is my submit.php code: [PASTE]

Please **simulate the logic** step-by-step. tell me if I am missing a connection, a variable name mismatch, or if I forgot to include the database file."

4. The "Check Engine Light"

By default, PHP often hides errors to look professional. When developing, you need to see everything.

If you see a blank white screen (The "White Screen of Death"), ask the AI for this:

The Prompt:

"My PHP page is blank. Give me the code snippet to **enable all error reporting** (display_errors) so I can see what is wrong. Tell me where to paste it in my file."

(Note for the Guide: The AI will give you ini_set('display_errors', 1); to put at the top of your script.)

5. Common "Silly" Mistakes to Watch For

Before you ask AI, check these three things. You will feel like a genius when you spot them yourself.

1. **The Missing Semicolon (;)**: In PHP, every command must end with a ;. It's like a period in a sentence. If you miss one, the whole page breaks.
2. **The Dollar Sign ($)**: All variables in PHP must start with $. $Name and Name are two different things to the computer.
3. **The Mismatch**: Did you name the input field email_address in HTML but ask for $_POST['email'] in PHP? They must match exactly.

6. Iterative Debugging

Sometimes the AI's fix causes a *new* error. This is normal!

Do not get discouraged.

- **You:** "Here is the error."
- **AI:** "Here is the fix."
- **You:** "Now I get a different error on line 50."
- **AI:** "Ah, that is because..."

Repeat this loop. It usually takes 2-3 tries to untangle complex code. This is not you failing; this is you **building**.

Summary of Charter 8

Debugging is 50% of the job. The best developers aren't the ones who never make mistakes; they are the ones who are good at finding them. With AI, you have the world's best debugger sitting next to you 24/7. Use it.

Charter 9: Polishing & Security – The Professional Touch

1. Security First

You are building tools for real businesses. You cannot leave doors open for hackers.

- **The Prompt:** ***"Review my code for SQL Injection vulnerabilities. Rewrite the query using Prepared Statements."***
- **The Prompt:** ***"Add htmlspecialchars() to the output to prevent XSS attacks."***

2. Making it Beautiful (UI/UX)

Your client judges the book by its cover. Even if the code is perfect, if it looks ugly, they won't pay.

- **Tailwind CSS: Do not write CSS from scratch. Ask the AI:** ***"Style this HTML table using Tailwind CSS. Give it a professional 'Dashboard' look with soft shadows and rounded corners."***

3. The "Readme" File

Always ask the AI to generate a documentation file for you.

- **The Prompt:** ***"Write a README.txt file explaining how to install this software on a new computer. Include database credentials and login details."***

Charter 10: Monetization – Turning Logic into Income

1. Stop Selling Code, Start Selling Solutions

The biggest mistake new developers make is trying to sell "PHP Scripts."

Business owners do not care about PHP, MySQL, or XAMPP. They care about **Time** and **Money**.

- **Don't sell:** "I can build a PHP database with a login form."
- **Do sell:** "I can build a system that tracks your inventory so you stop losing stock."
- **Do sell:** "I can build a portal that lets parents check their child's exam results on their phone, saving your teachers 10 hours a week."

You are not a coder. You are a **Problem Solver**.

2. Model A: The "Local MVP" (Minimum Viable Product)

You don't need to build the next Facebook. You just need to build tools for your local community.

Look around your town. Who is struggling with paper records?

- **The Hardware Store:** They write sales in a book. Build a simple "Point of Sale" (POS) system where they enter the item and price, and it saves to a database.
- **The Clinic:** They lose patient files. Build a "Patient Record System" (CRUD: Create Patient, Search Patient, Add History).
- **The Car Wash:** They want to track loyal customers. Build a simple tracking app.

How to Price:

Don't charge hourly. Charge per project. A simple inventory system is worth **KES 15,000 - 30,000** to a business owner if it stops them from losing money.

3. Model B: The "Reskinning" Strategy (High Profit)

This is the smartest way to work.

You build **one** great application, and you sell it to **ten** different clients.

The Strategy:

1. Use AI to build a solid **School Management System** (Features: Student Registration, Fee Recording, Exam Results).
2. **Client 1:** Sell it to "St. Mary's High School." Add their logo and green colors.
3. **Client 2:** Sell the *exact same code* to "Nairobi Academy." Change the logo and make the colors match their logo.
4. **Client 3:** Sell it to a local college.

Why this works: You do the hard work once. Every sale after the first one is 100% profit.

4. Model C: The "Maintenance" Retainer

Selling the software is a one-time payment. But you want **recurring income**.

Offer a "Support Contract."

- **The Pitch:** "For KES 2,000 per month, I will ensure the system stays online, back up your database every week, and fix any bugs that appear."
- If you have 10 clients paying KES 2,000/month, that is **KES 20,000/month** guaranteed income for very little work.

5. How to Market Yourself (Using AI)

You don't need to be a marketing expert. Use the AI to write your proposals.

The Proposal Prompt:

"I want to pitch a [Inventory Management System] to a local [Pharmacy] owner.

Write a persuasive email proposal.

- **Problem:** Acknowledge that tracking drugs on paper causes errors and stock theft.
- **Solution:** Offer a simple, secure digital system.
- **Tone:** Professional, respectful, and business-focused.
- **Call to Action:** Ask for a 15-minute demo meeting."

6. Where to Find Clients?

1. **Walk-ins:** Literally walk into local businesses. "I noticed you are recording sales in a book. Can I show you a demo of a digital version I built?"
2. **WhatsApp Status:** Post screenshots of your apps. "Just built this system for a client. DM me if you need something similar."
3. **Offer a Free Trial:** "I'll install this on your laptop. Use it for one week. If you don't like it, I'll remove it. If you like it, pay me." (90% of people will pay because they won't want to go back to paper).

Final Words of Advice

The barrier to entry has never been lower. In the past, you needed 4 years of Computer Science to build these tools. Today, you just need a clear idea, a laptop, and the ability to prompt the AI.

Start small. Build something simple. Sell it. Repeat.